

Keep a Shell Open in NodeJS and Reuse for Multiple Commands

 Full Post Details

Date Posted:	Nov. 08, 2020
Last Updated:	Nov. 08, 2020

 Tags

Tags: cli javascript nodejs

I'm making this post because this question has been bugging me, and I have had an unreasonably difficult time finding information on it. My question is simple:

Is there a way to spawn a shell in NodeJS, *keep it open*, and reuse it for multiple commands (perhaps even calling different binaries)?

Throughout this post, I'll use this as the sample scenario:

1. Spawn a shell
2. Execute `uname -a` to get OS Info
3. Execute `ls -a` to list files in the current directory

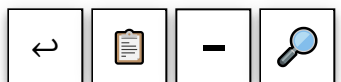
Without Reuse

The standard approach is to not care about re-using a process or shell and letting NodeJS just spawn & kill them automatically. For example:



```
const childProcess = require('child_process');
const osInfo = childProcess.execSync('uname -a').toString();
const files = childProcess.execSync('ls -a').toString();
console.log({
  osInfo,
  files
});
```

If you wanted an async based approach, you could use something like:

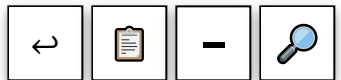


```
const childProcess = require('child_process');

const osInfo = await new Promise((res, rej) => {
  childProcess.exec('uname -a', (err, out) => res(out));
});
const files = await new Promise((res, rej) => {
  childProcess.exec('ls -a', (err, out) => res(out));
});
console.log({
  osInfo,
  files
});
```

Spawning an Actual Shell Process

With most of the `child_process` commands you can always specify that a command should execute inside of a shell:



```
require('child_process').execSync('uname -a', {
  shell: true
});
```

```
});
```

In fact, you can even pass in whatever shell you want used, as a path:



```
// This will fail on Windows
require('child_process').execSync('uname -a', {
  shell: '/bin/sh'
});
```

However, the shell that is spawned by `execSync`, `exec`, etc., is sort of *managed* by the method, and not exposed directly. If your command is `ls`, even with `shell: true`, you are still piping commands to `ls`, not to the underlying shell. However... what if you spawn the shell *as* your command? That works!



```
const spawnedShell = require('child_process').spawn('/bin/sh');
// Capture stdout
spawnedShell.stdout.on('data', d => console.log(d.toString()));
// Write some input to it
spawnedShell.stdin.write('uname -a\n');
// We can add more commands, but make sure to terminate
spawnedShell.stdin.write('ls -a\n');
// End
spawnedShell.stdin.end();
```

This is pretty neat! However, it is not super portable (`/bin/sh` is certainly not going to work on Windows), and it is not really well-suited as a persistent shell that we can use in a nice async manner. Let's refine it.

Reusing an Open Shell in NodeJS

The generic steps we can use to set up a long-lived shell to pipe data in and out of in NodeJS is:

- Spawn the shell directly with `child_process.spawn()`
 - Optimal solution would automatically use the correct shell based on OS
- Add event listeners / hook into the subprocess *streams*
 - At a minimum, we want to capture `stdout`, catch `exit`(s), and write to `stdin`
- Allow results to be awaited

With this in mind, I put together a tiny wrapper around `child_process.spawn`, which launches [the correct shell](#) and exposes some utility methods. You can find it [here](#), but keep in mind it is not optimized and comes with some caveats.

Performance

The main reason why I was looking into this in the first place was that I was wondering if there would be any performance benefits to keeping a shell open and reusing it, rather than launching an entirely new process for every command. I'm working on a project that tries to execute hundreds of commands, so any increase in speed is something I'm interested in.

Unfortunately, based on my [simplistic test](#), reusing a shell actually is *less performant* than just allowing NodeJS to spawn an entirely new process for each command. Sample results (200 commands executed):

Method	Total Time (ms) (lower is better)	Ops per Sec
<code>spawn</code>	1892	105.7
<code>spawnWithShell</code>	2579	77.5

Method	Total Time (ms) (lower is better)	Ops per Sec
<code>exec</code>	2634	75.9
<code>persistentShellWithCapture</code> (**)	5008	39.9
<code>spawnSync</code>	5399	37.0
<code>persistentShell</code> (**)	5770	34.7
<code>execSync</code>	8982	22.3

** = `persistentShell` and `persistentShellWithCapture` are the wrappers I created around `spawn`, which spawn an instance of the default OS shell and pipe commands and data in and out.

This test was performed on a laptop running Windows 10 and Node v12.x

As you can see, `child_process.spawn` easily performed the best out of all the approaches. This makes a lot of sense, as pretty much all the other approaches build *on-top of* spawn, so by definition they are adding overhead. In fact, the idea that `child_process` methods are all extending from `.spawn` is called out in the very first paragraph of [the official docs on `child\_process`](#):

“[...] This capability is primarily provided by the `child_process.spawn()` function:”

At first, I was a little surprised at just how poorly my shell-reuse technique fared, but in hindsight, it makes sense. Even though the shell is reused, it still has to be launched (initially), and a communication layer maintained to pipe stdin/stdout. Furthermore, any command that is executed *still* requires the running of the process to support it; whether you run `git log` via a spawned shell or with Node's `spawn`, both cases require the git binary to actually run, and in many cases, spawning and keeping open a shell wrapper around those commands is pretty much pure overhead.

Other Approaches? Libraries?

So, although you definitely *can* keep a shell open in NodeJS and pipe things in and out, in general it is not a great idea for both security and performance purposes.

So, back to the original reason why I was researching this; is there a *different* way to speed up slow command execution, especially in bulk? Well, I might have to get back to you, but I have a feeling that another avenue I need to explore more is native bindings.

For example, `git` exposes commands through its binary (which you can call through `child_process` or your CLI), but there are also packages that expose *native bindings* for Node, where Node talks directly to the low-level code of the program, rather than going through a CLI or spawning processes to execute the commands. In general, talking directly to lower-level (aka nearer to *bare metal*) code is going to be faster than passing messages through higher level interfaces.

A very interesting article I came across (of course, *after* I had already written most of this post), is by Github's engineering team: "[Integrating Git in Atom](#)". They switched *from* a native binding package (`nodegit`), to [executing git commands via spawn](#) (via `child_process.spawn` / `execFile`). They noted that there was a significant performance *decrease* caused by the overhead in process spawning per command, although they also noted some ways they helped mitigate the effect. This seems to confirm my point though, that native bindings are going to be more performant than spawning a new process for each command.

Links of Interest

These are links I came across while researching this topic, and that were helpful:

- [NodeJS source code](#)
- github.com/nodejs/help/issues/1183
- Stack Overflow: [Exec – Keep Shell Alive](#)

- Stack Overflow: [Spawn vs Execute](#)
- FreeCodeCamp: [NodeJS Child Processes – Everything You Need to Know](#)

Donate

joshuatz.com/support-me

Follow Me Elsewhere

linkedin.com/in/joshuatzucker

One thought on “Keep a Shell Open in NodeJS and Reuse for Multiple Commands”



Soldeplata says:

[August 24, 2022 at 11:06 pm](#)

Thanks Joshua. This was actually useful. I changed all my ``execSync`` for ``spawnSync`` and I saw a performance improvement.

Reply

© Joshua Tzucker. Code snippets are covered under [MIT License](#), unless otherwise noted.